

GNR-Q: Inference-Time State-Conditioned Reconstruction for Memory-Efficient Quantized Language Models

SeungGeun Baeck¹ Dumipyo² HaeJung Suk^{3,*}

¹Vhexlab Co., Ltd.

²Department of Psychology, Ajou University

³Department of Digital Media, Ajou University

undeturmoil@gmail.com dumipyo@gmail.com dbdip@ajou.ac.kr

*Corresponding author

September 2026

Abstract

Low-bit quantization substantially reduces the memory footprint of large language models (LLMs), but the associated loss of numerical precision can distort internal representations and degrade downstream prediction quality. We investigate whether part of this lost representation quality can be reconstructed dynamically at inference time rather than preserved explicitly in model weights. We introduce **GNR-Q** (Guided Neural Reconstruction for Quantization), an inference-time state-conditioned reconstruction framework in which a compact learned sidecar observes a hidden representation produced by a frozen quantized model and predicts a residual correction toward the corresponding full-precision representation.

Using Qwen3-4B-Base, we first perform controlled differentiable W4A16 and W3A16 experiments to study reproducibility, layer placement, and reconstruction capacity. We then evaluate an actual TorchAO packed W4 model. Packing 252 decoder linear modules reduces measured CUDA-allocated model memory from 7.545 GiB in BF16 to 2.792 GiB, a 63.0% reduction and a $2.70\times$ compression ratio. A 0.986M-parameter GNR-Q sidecar requires only 1.88 MiB in BF16. When trained exclusively to reconstruct the BF16 hidden representation, with no next-token cross-entropy supervision, GNR-Q reduces final hidden-state MSE from 0.5376 to 0.3901 and improves perplexity from 15.000 to 14.415, recovering 27.52% of the cross-entropy degradation introduced by quantization. A parameter-matched low-rank projection control recovers 26.51%, suggesting that much of the predictable residual has compact state-dependent structure.

These results provide evidence that some representation fidelity removed by low-bit quantization can be replaced by lightweight inference-time computation, establishing a practical memory–compute trade-off for quantized LLM inference.

1 Introduction

Large language models increasingly face deployment constraints determined not only by computational throughput but also by memory capacity and memory bandwidth. Low-bit quantization addresses these constraints by representing model parameters with fewer bits, enabling substantially larger models to execute within a fixed device-memory budget. Modern post-training quantization (PTQ) methods have demonstrated that transformer weights can often be reduced to 4-bit and,

in some settings, 3-bit or lower precision while retaining much of the original capability [1, 2]. Nevertheless, quantization is not lossless. Replacing full-precision parameters with low-bit approximations alters the transformations performed throughout the network, producing deviations in hidden representations and ultimately in next-token probabilities.

Most existing methods mitigate this degradation by improving the quantizer itself, preserving important channels, calibrating layer-wise error, or storing additional low-rank correction parameters [3–6]. This paper studies a complementary possibility: *can some of the representation fidelity removed by quantization be reconstructed only when inference is performed?*

Let h_t^{FP} denote a full-precision hidden representation and h_t^Q the corresponding representation produced by a quantized copy of the same pretrained model. Quantization induces the representation residual

$$e_t = h_t^{\text{FP}} - h_t^Q. \quad (1)$$

Instead of treating e_t only as damage to be prevented before deployment, we ask whether a compact learned function can estimate part of this unavailable residual from the quantized state itself:

$$\hat{e}_t = R_\theta(h_t^Q), \quad \hat{h}_t = h_t^Q + \hat{e}_t. \quad (2)$$

We call this framework GNR-Q. The full-precision model is used only as an offline teacher. During deployment, the teacher and the true residual are absent; the sidecar must estimate the missing counterfactual full-precision state from the surviving quantized representation.

This formulation introduces a memory–compute trade-off:

$$\text{stored precision} \longrightarrow \text{low-bit backbone} + \text{lightweight reconstruction compute}. \quad (3)$$

The conceptual motivation is related to neural reconstruction strategies used in other computational domains: instead of explicitly retaining every high-fidelity detail, a lower-cost representation is supplemented by learned inference. Our claim is not that quantization can be exactly inverted, nor that discarded bits remain literally encoded in the quantized model. Rather, we test whether the surviving state contains sufficient statistical structure to estimate a useful component of the full-precision counterfactual.

Recent mechanistic work supports the possibility that quantization error is structured. Chen et al. show that hidden-state errors in pretrained quantized LLMs do not simply accumulate independently through depth; newly introduced errors often partially oppose inherited errors, slowing error growth [8]. This motivates viewing quantization error as part of a learned representation trajectory rather than as independent additive noise.

Our experiments support four main findings. First, reconstruction is reproducible across random seeds and remains effective under more severe W3 quantization. Second, reconstruction quality depends systematically on insertion layer and sidecar capacity. Third, in a real packed W4 model, a 1.88 MiB sidecar recovers 27.52% of the quantization-induced cross-entropy gap while preserving approximately 63% lower CUDA-allocated model memory than BF16. Fourth, a parameter-matched low-rank projection control recovers almost the same amount, suggesting that much of the recoverable residual has compact state-predictable structure.

The purpose of this paper is therefore not to introduce a universally superior quantizer. Instead, we validate a distinct inference-time reconstruction formulation in which a frozen low-bit model exposes enough structure in its current hidden state for a compact learned sidecar to estimate part of the unavailable full-precision representation.

2 Related Work

2.1 Post-Training Quantization for LLMs

GPTQ introduced an efficient second-order one-shot weight-quantization framework for very large generative models [1]. AWQ later showed that activation-aware scaling and selective protection of salient weight channels can provide hardware-efficient low-bit inference without reconstruction-based retraining [2]. Recent empirical work on Qwen3 shows that moderate bit widths remain usable but that degradation becomes pronounced in lower-bit regimes, especially at 3 bits and below [7].

These methods primarily seek a better low-bit parameterization. GNR-Q instead assumes that a quantized backbone already exists and asks whether residual representation degradation can be corrected during inference.

2.2 Static Low-Rank Quantization-Error Correction

A closely related line of research augments quantized models with low-rank correction terms. LQER uses low-rank approximation together with activation-induced scaling to reconstruct quantization error [3]. QERA derives analytical low-rank reconstruction terms by minimizing expected output error rather than only raw weight error [4]. ASER combines low-rank error reconstruction with activation smoothing [5]. More recent approaches such as GlowQ and BALANCER focus on reducing the memory and latency overhead of low-rank correction or distributing correction capacity under explicit budgets [12, 14]. SandwichQuant identifies an efficient normalization-affine correction subspace before and after quantization [13].

These methods establish that quantization error often has exploitable low-dimensional structure. The principal difference is where the correction is represented. Static correction methods attach learned or analytically derived parameters to weight-space or layer-space transformations. In GNR-Q, the auxiliary parameters define a *function* that generates a residual correction from the current runtime hidden state. The correction itself is not stored per token or per inference state.

2.3 Activation- and Representation-Space Correction

Low-Rank Correction (LRC) introduces full-precision low-rank matrices acting on available unquantized activations to compensate for joint weight-and-activation quantization error [9]. IMPACT similarly emphasizes that activations may exhibit stronger low-rank structure than model weights and derives importance-aware reconstruction bases in activation space [10].

Residual Fallback Quantization (RFQ) is a particularly close conceptual neighbor. RFQ explicitly computes the residual $r = x - Q(x)$ because the original activation x is available immediately before activation quantization, and it preserves that residual through an auxiliary low-bit pathway [11]. GNR-Q addresses a different condition: at deployment, the corresponding full-precision hidden state is unavailable. The sidecar must therefore predict an unavailable counterfactual residual from h^Q alone. RFQ encodes an available residual; GNR-Q estimates a missing one.

2.4 Error Propagation and Representation Drift

Quantization Error Propagation (QEP) explicitly models accumulation of layer-wise quantization error during calibration, allowing downstream layers to compensate for inherited error [6]. Chen et al. provide a complementary mechanistic analysis, showing that pretrained LLMs naturally exhibit partial error cancellation across blocks and LM-head geometry that attenuates some hidden-state

perturbations [8]. These findings support the view that quantization error is a network-level dynamic process.

In the literature reviewed through September 2026, we found substantial work on low-rank weight correction, activation reconstruction, residual preservation, calibration-time error propagation, and post-quantization parameter adaptation. We did not identify the same deployment formulation in which a frozen low-bit language model emits a hidden state and a compact sidecar predicts the unavailable full-precision hidden residual solely from that quantized state. We therefore position GNR-Q as a complementary inference-time reconstruction mechanism rather than as a replacement for established PTQ methods.

3 Method

3.1 Problem Formulation

Let F^{FP} be a pretrained model in full precision and F^Q a quantized copy originating from the same checkpoint. At transformer layer l , define

$$h_l^{\text{FP}} = F_l^{\text{FP}}(x), \quad h_l^Q = F_l^Q(x), \quad (4)$$

and the quantization-induced representation residual

$$e_l = h_l^{\text{FP}} - h_l^Q. \quad (5)$$

The reconstructor R_θ predicts

$$\hat{e}_l = R_\theta(h_l^Q), \quad \hat{h}_l = h_l^Q + \hat{e}_l. \quad (6)$$

Full-precision states are used only during offline training.

3.2 GNR-Q Reconstructor

For hidden width d and bottleneck rank r , the primary reconstructor is a compact gated low-rank network. First,

$$u_l = \text{RMSNorm}(h_l^Q). \quad (7)$$

A down-projection creates two rank- r components:

$$[a_l, b_l] = W_{\text{down}} u_l, \quad W_{\text{down}} \in \mathbb{R}^{2r \times d}. \quad (8)$$

The bottleneck is gated using

$$z_l = \text{SiLU}(a_l) \odot b_l, \quad (9)$$

and the predicted residual is

$$\hat{e}_l = W_{\text{up}} z_l, \quad W_{\text{up}} \in \mathbb{R}^{d \times r}. \quad (10)$$

The total trainable parameter count is approximately

$$N_R = d + 3dr. \quad (11)$$

For Qwen3-4B-Base, $d = 2560$ and $r = 128$, giving

$$N_R = 985,600, \quad (12)$$

or approximately 0.986M parameters and 1.88 MiB in BF16.

3.3 Reconstruction Objective

The primary objective is normalized hidden-state reconstruction:

$$\mathcal{L}_{\text{rec}} = \frac{\text{MSE}(\hat{h}_l, h_l^{\text{FP}})}{\text{MSE}(h_l^Q, h_l^{\text{FP}}) + \epsilon}. \quad (13)$$

Both teacher and quantized backbone remain frozen; only R_θ is optimized.

The principal packed-model experiment uses no token-level objective:

$$\mathcal{L} = \mathcal{L}_{\text{rec}}. \quad (14)$$

An auxiliary task-aware variant additionally uses

$$\mathcal{L} = \mathcal{L}_{\text{rec}} + \lambda \mathcal{L}_{\text{CE}}, \quad \lambda = 0.25, \quad (15)$$

but this variant is not used as the primary evidence for pure quantization-loss recovery.

3.4 Diagnostic Intermediate-Layer Protocol

For mechanistic experiments we use differentiable simulated W4A16 and W3A16 quantization. Linear weights are group-wise quantized with symmetric round-to-nearest quantization using group size 128 and then dequantized into BF16 tensors. This reproduces quantization-induced numerical error while retaining differentiability; it does *not* physically reduce model memory.

For reconstruction at layer l , the quantized network is split into prefix and suffix:

$$F^Q = F_{>l}^Q \circ F_{\leq l}^Q. \quad (16)$$

The quantized prefix produces h_l^Q , reconstruction produces $\hat{h}_l$, and the frozen suffix computes

$$\hat{h}_L = F_{>l}^Q(\hat{h}_l). \quad (17)$$

Training consists of local reconstruction followed by one optional frozen-suffix refinement epoch using normalized final-hidden-state MSE.

3.5 Actual Packed W4 Protocol

The deployment-oriented experiment uses TorchAO 0.18.0 weight-only INT4 quantization with group size 128, HQQ parameter selection, BF16 activations, and the `tile_packed_to_4d` storage format. A total of 252 decoder `Linear` modules are converted to `Int4TilePackedTo4dTensor`.

In our environment, the packed operation `aten::_weight_int4pack_mm` supports forward inference but does not implement the activation-input derivative. We therefore treat the packed backbone as strictly frozen. Teacher and packed-student final hidden states are generated under no-gradient execution and detached. Reconstruction is then learned at the final hidden state immediately before the original frozen LM head:

$$h_L^Q \rightarrow R_\theta(h_L^Q) \rightarrow \hat{h}_L \rightarrow W_{\text{LM}} \hat{h}_L. \quad (18)$$

The teacher is absent during deployment.

3.6 Parameter-Matched Low-Rank Projection Control

To isolate the role of nonlinear gating, we use a parameter-matched low-rank projection control:

$$\hat{e} = W_2 W_1 \text{RMSNorm}(h^Q). \quad (19)$$

With bottleneck rank 192, its parameter count is

$$N_{\text{ctrl}} = d + 2d(192) = 985,600, \quad (20)$$

exactly matching the rank-128 GNR-Q sidecar. Because RMSNorm is present, we refer to this baseline as a low-rank projection control rather than a strictly linear model.

3.7 Memory Accounting

Let M_{FP} be BF16 model memory, M_Q packed quantized model memory, and M_R sidecar memory. For the actual packed comparison, model memory refers to the PyTorch CUDA allocation reported by `torch.cuda.memory_allocated()` after model construction or quantization; it is not a measurement of total physical or system-wide unified memory. Deployment memory is

$$M_{\text{deploy}} = M_Q + M_R, \quad (21)$$

and relative memory reduction is

$$S = 1 - \frac{M_Q + M_R}{M_{\text{FP}}}. \quad (22)$$

4 Experimental Setup

4.1 Model, Hardware, and Software

All experiments use Qwen3-4B-Base [15], checkpoint revision 906bfd4b4dc7f14ee4320094d8b41684abff8539. Experiments run on a single NVIDIA DGX Spark equipped with an NVIDIA GB10 GPU (compute capability 12.1). The software environment is PyTorch 2.13.0+cu130, CUDA 13.0, Transformers 5.17.0, Datasets 5.0.1, TorchAO 0.18.0, and Python 3.12.3. All research dependencies are isolated in Docker containers.

4.2 Dataset

We use WikiText-2 [16]. The training split is used for reconstruction training and the validation split for evaluation. Text is tokenized with the Qwen3 tokenizer and partitioned into fixed 128-token blocks. Primary runs use 256 training blocks and 64 validation blocks. Training uses batch size 4 and evaluation uses batch size 2. The calibration set therefore contains 32,768 training tokens. The present implementation concatenates each split before dividing it into blocks; the resulting tokenizer warning about the concatenated corpus length does not affect the actual 128-token model inputs.

4.3 Optimization

For simulated experiments, both local reconstruction and frozen-suffix refinement use AdamW with weight decay 0.01 and gradient clipping to maximum global norm 1.0. Local reconstruction uses learning rate 3×10^{-4} , while frozen-suffix refinement uses 0.3 times that rate, i.e., 9×10^{-5} in the reported experiments. No learning-rate scheduler is used. Default simulated W4 uses layer 27, rank

Experiment	Quantization	Location	Rank	Seeds
W4 reproducibility	simulated W4A16	layer 27	128	1, 7, 42
W3 reproducibility	simulated W3A16	layer 27	128	1, 7, 42
Layer ablation	simulated W4A16	9, 18, 27	128	42
Rank ablation	simulated W4A16	layer 27	32–256	42
Packed pure GNR-Q	actual INT4	final state	128	42
Low-rank control	actual INT4	final state	192	42
Task-aware auxiliary	actual INT4	final state	128	42

Table 1: Experimental matrix. Simulated quantization is used for differentiable mechanistic analysis; actual packed INT4 is used for physical memory validation.

Seed	Final MSE	Cosine	Loss	PPL	Gap closure
Baseline	0.69849	0.96823	2.76287	15.845	0.00%
1	0.51430	0.97655	2.72689	15.285	18.03%
7	0.51534	0.97648	2.72581	15.269	18.57%
42	0.51626	0.97644	2.72731	15.292	17.83%

Table 2: Simulated W4A16 reconstruction across random seeds. Mean gap closure is 18.14% with sample SD 0.38 percentage points.

128, three local epochs, one suffix-refinement epoch, and seed 42; because the script default is two local epochs, the released reproduction commands explicitly set `--local-epochs 3`.

The packed experiment uses AdamW with the same learning rate and weight decay, gradient clipping at 1.0, four reconstruction epochs, rank 128, and seed 42.

4.4 Experimental Matrix

4.5 Metrics

We report hidden-state MSE, hidden-state cosine similarity, next-token cross-entropy, and perplexity. The main downstream recovery measure is cross-entropy gap closure:

$$G_{\text{CE}} = \frac{L_Q - L_R}{L_Q - L_{\text{FP}}} \times 100, \quad (23)$$

where L_{FP} , L_Q , and L_R denote BF16, quantized, and reconstructed validation loss, respectively.

5 Results

5.1 Reproducibility Under Simulated W4 Quantization

The W4 baseline has validation loss 2.76287 and perplexity 15.845, compared with BF16 loss 2.56338 and perplexity 12.980. Across three seeds, GNR-Q consistently improves both hidden-state fidelity and downstream loss.

Mean final-hidden MSE reduction is approximately 26.23%. The narrow seed-to-seed variation indicates that the effect is not driven by a single favorable initialization.

Seed	Final MSE	Cosine	Loss	PPL	Gap closure
Baseline	4.95136	0.79155	5.51381	248.094	0.00%
1	2.83481	0.87329	5.08427	161.462	14.56%
7	2.85708	0.87290	5.09428	163.086	14.22%
42	2.81919	0.87369	5.06959	159.109	15.06%

Table 3: Simulated W3A16 reconstruction. Mean gap closure is 14.61% with sample SD 0.42 percentage points.

Layer	Local MSE reduction	Final MSE	Final reduction	PPL	Gap closure
9	49.87%	0.52090	25.42%	15.059	25.50%
18	15.77%	0.51404	26.41%	15.121	23.46%
27	12.16%	0.51626	26.09%	15.292	17.83%

Table 4: Layer-placement ablation under simulated W4A16.

5.2 Severe W3 Quantization

Under simulated W3A16 quantization, perplexity rises from 12.980 to 248.094 and final hidden-state MSE rises to 4.95136. Reconstruction remains effective but cannot fully restore the heavily degraded trajectory.

Mean final-hidden MSE reduction is approximately 42.70%, much larger than the downstream gap closure. This divergence suggests that not every component of geometric representation error is equally important for next-token prediction.

5.3 Reconstruction Placement Matters

Among the tested positions, layer 9 gives the strongest downstream recovery, although layer 18 gives slightly lower final hidden-state MSE. This shows that Euclidean representation similarity and task-relevant recovery are related but not interchangeable.

5.4 Capacity–Recovery Relationship

Recovery improves steadily through rank 128 but largely saturates in downstream cross-entropy between ranks 128 and 256, even though hidden-state MSE continues to improve. Rank 128 therefore provides a favorable quality-to-sidecar-size operating point in this setup.

5.5 Actual Packed W4 Memory and Quality

The central deployment experiment uses actual packed INT4 weights. Measured CUDA-allocated model memory falls from 7.545 GiB in BF16 to 2.792 GiB in packed W4, a 63.00% reduction and $2.70\times$ compression. The BF16 GNR-Q sidecar uses only 1.880 MiB, yielding a combined footprint of approximately 2.794 GiB and preserving approximately 62.97% lower CUDA-allocated model memory than BF16.

Pure representation reconstruction reduces hidden-state MSE by approximately 27.44% and closes 27.52% of the cross-entropy gap. Because no next-token objective is used, the downstream improvement arises as a consequence of moving the quantized hidden state toward its BF16 counterpart.

Rank	Params	BF16 storage	Final MSE	PPL	Gap closure
32	0.248M	0.47 MiB	0.54021	15.462	12.28%
64	0.494M	0.94 MiB	0.53129	15.396	14.42%
128	0.986M	1.88 MiB	0.51626	15.292	17.83%
256	1.969M	3.75 MiB	0.50268	15.280	18.19%

Table 5: Reconstructor rank ablation at layer 27 under simulated W4A16.

Configuration	CUDA alloc.	Sidecar	Loss	PPL	Hidden MSE	Gap closure
BF16	7.545 GiB	–	2.56338	12.980	–	–
Packed W4	2.792 GiB	–	2.70808	15.000	0.53760	0.00%
W4 + GNR-Q	~2.794 GiB	1.88 MiB	2.66825	14.415	0.39009	27.52%
W4 + low-rank control	~2.794 GiB	1.88 MiB	2.66972	14.436	0.39953	26.51%

Table 6: Main packed W4 results. The principal GNR-Q model is trained only on hidden-state reconstruction, with no next-token loss.

The low-rank projection control closes 26.51% of the gap with the same parameter count and storage. The nonlinear GNR-Q architecture is consistently better, but the small difference indicates that much of the recoverable residual can be represented by a compact state-conditioned low-rank mapping.

5.6 Task-Aware Auxiliary Variant

When a next-token objective is added with $\lambda = 0.25$, the packed model reaches loss 2.48053 and perplexity 11.948, lower than the BF16 reference. This result is not interpreted as 157% quantization-loss recovery: the sidecar is now directly optimized for language modeling and can move away from the BF16 representation if that improves the corpus-specific objective. We therefore treat this experiment as evidence that the same sidecar can also act as a tiny adaptation module, not as part of the primary reconstruction claim.

6 Discussion

6.1 Quantization Error Contains a Predictable Component

The central result is incompatible with a model in which quantization error is entirely independent, state-unrelated noise. If

$$e = h^{\text{FP}} - h^Q \quad (24)$$

were conditionally unpredictable from h^Q , a compact predictor would provide little benefit. Instead, the observed packed-model recovery implies that a meaningful component of the residual behaves as a statistically predictable function of the surviving representation.

A useful decomposition is

$$e = e_{\text{pred}} + e_{\text{irr}}, \quad (25)$$

where the optimal deterministic squared-error predictor targets

$$e_{\text{pred}} \approx \mathbb{E}[e \mid h^Q]. \quad (26)$$

This is a conditional-estimation claim, not an exact inversion claim. Quantization can irreversibly collapse multiple full-precision values into the same low-bit representation. GNR-Q does not recover discarded bits; it estimates a useful counterfactual full-precision state from statistical structure learned offline.

6.2 Compact Structure Dominates Architectural Complexity

The parameter-matched low-rank control recovers nearly the same cross-entropy gap as the nonlinear sidecar. This suggests that the primary phenomenon is not the expressive power of a large nonlinear network. Instead, much of the predictable residual appears accessible through a compact low-rank state-conditioned mapping. The current experiment does not directly prove that residual covariance is low rank; spectral analysis would be required for that stronger claim. We therefore describe the result as evidence for *compact* or *low-dimensional* predictable structure rather than as a measured intrinsic-rank theorem.

The more important distinction is state conditioning. Even the low-rank control produces

$$\hat{e}_t = W_2 W_1 \text{RMSNorm}(h_t^Q), \quad (27)$$

so different runtime states receive different corrections. This differs conceptually from storing a fixed weight-space residual.

6.3 Quantization Error Is Part of a Dynamic Trajectory

Layer placement substantially changes cross-entropy recovery. The strongest tested result occurs at layer 9, not at the latest tested intermediate layer. At the same time, final MSE values across layers are similar. A correction therefore matters not only because of how much error it removes locally, but also because of how it changes the subsequent transformer trajectory. This interpretation is consistent with recent evidence that inherited and newly introduced quantization errors interact structurally across pretrained transformer blocks [8].

We do not conclude that earlier correction is universally better. Very early states may not yet contain sufficient accumulated error, whereas very late states may leave too little downstream computation for corrections to affect task-relevant geometry. Reconstruction placement should instead be treated as an optimization variable.

6.4 Representation Fidelity and Task Fidelity Are Not Identical

Several results reveal a consistent distinction between geometric and functional reconstruction. Under W3, MSE recovery is much larger than cross-entropy recovery. Increasing rank from 128 to 256 improves MSE but barely improves cross-entropy. Layer 18 yields slightly better final MSE than layer 9 but worse language-model loss. Finally, the task-aware sidecar produces better-than-BF16 corpus loss despite having higher MSE than the pure reconstruction model.

These observations suggest that hidden-state error contains directions with very different functional importance. A future objective may therefore weight residual directions according to downstream sensitivity rather than optimizing unweighted MSE alone.

6.5 A Practical Memory–Compute Trade-Off

Packed W4 removes approximately 4.753 GiB of measured CUDA-allocated model memory relative to BF16, while the deployed sidecar reintroduces only 1.880 MiB. The saved-memory-to-sidecar-

storage ratio is therefore approximately 2,589:1. This makes the parameter-memory side of the trade-off strongly favorable.

However, memory is only one deployment resource. The present study does not measure end-to-end latency, decoding throughput, prefill throughput, or energy. Approximately one million additional low-rank multiply-accumulate operations per token is small relative to a 4B-parameter backbone, but kernel-launch and memory-movement overhead can dominate small modules. We therefore claim a demonstrated memory–additional-compute design space, not yet a proven latency advantage.

6.6 Relationship to Neural Reconstruction in Graphics

The original motivation came from the general computational strategy behind modern neural reconstruction systems: represent or compute a lower-cost intermediate state, then infer some missing high-fidelity detail only when needed. The analogy should not be taken as architectural equivalence to DLSS. Graphics reconstruction may use spatial, temporal, motion, or other guidance signals; the current GNR-Q prototype uses only the present quantized hidden state. The transferable principle is narrower: some fidelity that would otherwise require more explicit representation can potentially be replaced by learned runtime computation.

A more explicitly guided future version could condition on quantization scales, activation RMS, outlier indicators, layer identity, attention statistics, uncertainty estimates, or previous reconstruction states:

$$\hat{e}_l = R_\theta(h_l^Q, g_l). \quad (28)$$

7 Limitations and Future Work

This work is an initial proof of concept and has several important limitations.

Single model family and dataset. All experiments use Qwen3-4B-Base and WikiText-2. Although training and validation splits are separate, both come from the same corpus family. Replication across other model families, scales, domains, and multilingual or code datasets is necessary before making universality claims.

Small calibration set. The sidecar is trained using only 32,768 tokens, but calibration-size scaling is not characterized. Future work should measure reconstruction quality as a function of calibration data volume.

Packed evaluation is W4 only. Simulated W3 results are mechanistic and do not demonstrate real W3 memory compression. Practical packed 3-bit kernels would enable evaluation of whether more aggressive compression plus reconstruction provides a superior memory–quality operating point.

Packed-kernel backward limitation. The evaluated TorchAO packed INT4 kernel does not provide the activation-input derivative, so actual packed intermediate-layer end-to-end refinement is not tested. Future kernels, local intermediate-state training, or differentiable surrogate-to-packed transfer could close this gap.

No matched comparison against state-of-the-art static correction. We do not claim superiority over QERA, LQER, ASER, GlowQ, BALANCER, or related methods. A key future experiment is a matched-budget comparison among static low-rank correction, state-conditioned reconstruction, and hybrid static–dynamic correction.

Compact structure is inferred, not directly measured. The low-rank control suggests a compact predictable residual, but we do not compute the singular-value spectrum or intrinsic dimensionality of residual matrices. Spectral analysis is needed to distinguish total residual rank, predictable residual rank, and task-relevant residual rank.

Limited layer and rank sweeps. Only three intermediate layers and four ranks are evaluated. Denser sweeps and sparse multi-layer reconstruction may provide better recovery per unit of additional computation.

No latency, throughput, or energy benchmark. Measured memory savings are real within the PyTorch CUDA allocation framework, but production efficiency requires end-to-end serving measurements, including prefill, autoregressive decode, batching, and power consumption.

Short context and weight-only quantization. All experiments use sequence length 128 and weight-only quantization with BF16 activations. Long-context behavior, joint weight–activation quantization, and KV-cache quantization remain open questions.

Statistical replication. Primary simulated W3/W4 results use three seeds. The low variability is encouraging, but stronger statistical inference should come from more seeds and, more importantly, replication across architectures and datasets.

Future work should formulate compression and reconstruction jointly. Given bit width b , sidecar rank r , and reconstruction locations $\mathcal{S}$, deployment could be optimized as

$$\min_{b,r,\mathcal{S}} \mathcal{L}_{\text{deploy}} \tag{29}$$

subject to memory and reconstruction-compute budgets. Such a formulation treats quantization not as a one-way removal of precision but as one component of a broader resource-allocation problem.

8 Conclusion

We investigated whether representation quality lost through low-bit language-model quantization can be partially reconstructed at inference time rather than preserved entirely as stored numerical precision. GNR-Q uses a compact state-conditioned sidecar to estimate a residual toward the corresponding full-precision hidden state while leaving the quantized backbone frozen.

Across simulated W4 and W3 experiments, reconstruction consistently reduces representation error and improves language-model quality. In an actual packed INT4 Qwen3-4B-Base model, CUDA-allocated model memory falls from 7.545 GiB to 2.792 GiB. A 1.88 MiB BF16 sidecar trained only on hidden-state reconstruction reduces final hidden-state MSE by 27.44% and recovers 27.52% of the quantization-induced cross-entropy degradation. A parameter-matched low-rank projection control recovers 26.51%, suggesting that much of the recoverable residual has compact state-dependent structure.

The strongest conclusion supported by the present experiments is deliberately narrow: a substantial component of quantization-induced representation degradation is statistically predictable from the surviving quantized hidden state and can be reconstructed by a very small learned sidecar while preserving nearly all of the memory savings of low-bit inference. This establishes a complementary design axis for quantized language models in which stored precision and runtime reconstruction capacity can be allocated jointly.

Code availability. The code, experiment launchers, machine-readable results, and arXiv source bundle are released at <https://github.com/undeturmoil/GNR-Q>.

References

- [1] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv:2210.17323*, 2022.
- [2] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. *Proceedings of Machine Learning and Systems*, 6, 2024.
- [3] C. Zhang, J. Cheng, G. A. Constantinides, and Y. Zhao. LQER: Low-rank quantization error reconstruction for LLMs. In *Proceedings of the 41st International Conference on Machine Learning*, PMLR 235:58763–58779, 2024. arXiv:2402.02446.
- [4] C. Zhang, J. T. H. Wong, C. Xiao, G. A. Constantinides, and Y. Zhao. QERA: An analytical framework for quantization error reconstruction. In *International Conference on Learning Representations*, 2025. arXiv:2410.06040.
- [5] W. Zhao, Y. Shi, X. Lyu, W. Sui, S. Li, and Y. Li. ASER: Activation smoothing and error reconstruction for large language model quantization. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(21):22822–22830, 2025. doi:10.1609/aaai.v39i21.34443. arXiv:2411.07762.
- [6] Y. Arai and Y. Ichikawa. Quantization error propagation: Revisiting layer-wise post-training quantization. In *Advances in Neural Information Processing Systems*, volume 38, 2025. doi:10.52202/085713-5080.
- [7] X. Zheng, Y. Li, H. Chu, Y. Feng, X. Ma, Z. Wang, J. Luo, J. Guo, H. Qin, M. Magno, and X. Liu. An empirical study of Qwen3 quantization. *Visual Intelligence*, 4:11, 2026. doi:10.1007/s44267-026-00114-4. arXiv:2505.02214.
- [8] Y. Chen, M. Beyer, J. Zhu, and J. Chen. Why does post-training quantization work? *arXiv:2609.11716*, 2026.
- [9] M. Scetbon and J. Hensman. Low-rank correction for quantized LLMs. *arXiv:2412.07902*, 2024.
- [10] M. M. Chowdhury, D. A. Asante, E. Chang, and Y. Li. IMPACT: Importance-aware activation space reconstruction. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, pages 3975–3992, 2026. doi:10.18653/v1/2026.acl-long.183.

- [11] T. Rahman, M. T. Jazi, Y. Peng, Z. Ma, A. D. Raju, Y. Wang, X. Huang, H. Y. Mak, S. Golestan, H. Le, Y. Dong, W. Guo, and Y. Wang. Activation outliers matter: Robust recovery for quantized multimodal LLMs. *arXiv:2608.26581*, 2026.
- [12] S. An, I. h. Suh, and Y. Kim. GlowQ: Group-shared low-rank approximation for quantized LLMs. *arXiv:2603.25385*, 2026.
- [13] P. Xia and J. Pang. SandwichQuant: Which parameters matter before and after quantization? *arXiv:2608.24173*, 2026.
- [14] S. Fan and Y. Lao. Budget-aware LLM quantization and low-rank correction via information-guided subspace matrices. In *Proceedings of the Thirty-Fifth International Joint Conference on Artificial Intelligence*, pages 4269–4277, 2026. doi:10.24963/ijcai.2026/475.
- [15] A. Yang et al. Qwen3 technical report. *arXiv:2505.09388*, 2025.
- [16] S. Merity, C. Xiong, J. Bradbury, and R. Socher. Pointer sentinel mixture models. *arXiv:1609.07843*, 2016.

A Auxiliary Task-Aware Result

For completeness, when the packed rank-128 sidecar is trained using

$$\mathcal{L} = \mathcal{L}_{\text{rec}} + 0.25\mathcal{L}_{\text{CE}}, \tag{30}$$

validation loss falls from the packed baseline 2.70808 to 2.48053, with perplexity 11.948. This is lower than the BF16 validation loss 2.56338. We do not interpret the corresponding nominal gap closure above 100% as quantization-loss recovery because next-token supervision permits corpus-specific adaptation beyond reconstruction of the BF16 state.

B Reproducibility Notes

The public repository accompanying this manuscript is intended to include the Dockerfiles, simulated and packed experiment scripts, shell commands for seed/layer/rank ablations, machine-readable JSON result files, and a source bundle matching the arXiv manuscript. Model weights and the WikiText-2 corpus are not redistributed; the scripts retrieve them from their original public sources.